

## Example 1: Traffic Flow Simulator

**Theme:** Model cars moving on a multi-lane road with traffic lights. Experiment with speed limits, light timing, and number of cars to analyze congestion and throughput (real-world traffic engineering simulation).

### Step 1: Design Document (Day 1–2 – submit first!)

**Problem Statement:** Simulate traffic flow over time steps. Users change parameters (number of cars, light cycle, speed) and see how it affects average speed and total cars that reach the destination. Results saved to CSV for analysis. **Classes:** Car (position, speed), Road (manages ArrayList of cars + simulation rules). **Pseudocode (you write this yourself – this is your computational thinking):**

```
runSimulation(int timeSteps, int numCars, int lightCycle) {  
    for each timeStep {  
        move all cars forward based on speed and road rules;  
        if traffic light is red, stop cars in certain lanes;  
        remove cars that reached the end;  
        log time, carsOnRoad, averageSpeed, throughput;  
    }  
}
```

### Step 2: Milestone 1 – Core Classes (Day 3 – in class)

**Car.java** (skeleton – fill in the TODOs after your Design Document)

```
public class Car {  
    private int position;  
    private double speed;    // 0.0 to maxSpeed  
  
    public Car(int startPosition, double startSpeed) {  
        // TODO: Assign values  
    }  
}
```

## APCSA Final Exam Project 2026

```
    public int getPosition() { return position; }

    public double getSpeed() { return speed; }

    public void setSpeed(double newSpeed) { /* TODO */ }

    public void move(int roadLength) { /* TODO: update position
*/ }

    public String toString() { /* TODO */ }

}
```

### Road.java (skeleton)

```
import java.util.ArrayList;
import java.io.*;

public class Road {

    private ArrayList<Car> cars = new ArrayList<>();
    private String logFile = "traffic_log.csv";
    private int roadLength = 1000; // meters
    private boolean lightIsGreen = true;

    public Road(int numCars, double initialSpeed) {
        // TODO: Create ArrayList of Car objects
    }

    public void runOneTimeStep(int lightCycle) {
        // TODO: Implement your pseudocode rules (loops +
if/else for light, movement, removal)
```

## APCSA Final Exam Project 2026

```
    }

    public void saveLog(int timeStep) {
        // TODO: Append current stats to CSV using PrintWriter
    }

    // TODO: Add analysis methods (getAverageSpeed(),
    getThroughput(), etc.)
}
```

### Step 3: Milestone 2 & Menu (Days 4–8)

**Main.java** (skeleton menu – expand after Milestone 2)

```
public class Main {
    public static void main(String[] args) {
        // TODO: Create Road object
        // Menu loop:
        // 1. Run simulation (ask for timeSteps, numCars,
        lightCycle)
        // 2. View traffic log
        // 3. Analyze results (e.g., "When did congestion
        peak?")
        // 5. Save & Quit
    }
}
```

Development Log Requirement: Paste every AI prompt + your modifications. Reflection Prompt: “How did changing the traffic light cycle affect throughput? What real-world traffic solution does this suggest?”

## Example 2: Stock Market Simulator

**Theme:** Simulate a simple stock market with multiple stocks. Users buy/sell shares and run daily price simulations based on volatility and trends. Analyze portfolio performance over time (computational finance modeling).

### Step 1: Design Document (Day 1–2 – submit first!)

**Problem Statement:** Simulate daily stock price changes and allow trading. Users experiment with different stocks and strategies; results and portfolio saved to CSV. **Classes:** Stock (name, price, volatility), Portfolio (manages ArrayList of stocks + trading + simulation). **Pseudocode (you write this yourself):**

```
runSimulation(int days, double marketTrend) {  
    for each day {  
        for each stock {  
            change price using volatility + random factor +  
marketTrend;  
        }  
        update portfolio value;  
        log day, each stock price, total portfolio value;  
    }  
}
```

### Step 2: Milestone 1 – Core Classes (Day 3 – in class)

**Stock.java** (skeleton)

```
public class Stock {  
    private String name;  
    private double currentPrice;  
    private double volatility; // 0.0 to 1.0  
  
    public Stock(String name, double initialPrice, double  
volatility) {  
        // TODO: Assign values
```

## APCSA Final Exam Project 2026

```
    }

    public double getCurrentPrice() { return currentPrice; }
    public void updatePrice(double marketTrend) {
        // TODO: Implement price change formula
    }

    public String toString() { /* TODO */ }
}
```

### Portfolio.java (skeleton)

```
import java.util.ArrayList;
import java.io.*;

public class Portfolio {
    private ArrayList<Stock> stocks = new ArrayList<>();
    private double cash = 10000.0;
    private String logFile = "market_log.csv";
    public Portfolio() {
        loadStarterStocks(); // TODO: implement
    }
    private void loadStarterStocks() {
        // TODO: Create initial stocks (or read from CSV)
    }
    public void runOneDay(double marketTrend) {
        // TODO: Update all stock prices using your pseudocode
    }
}
```

## APCSA Final Exam Project 2026

```
    public void buyStock(String name, int shares) { /* TODO */ }

    public void sellStock(String name, int shares) { /* TODO
*/ }

    public void saveLog(int day) {
        // TODO: Append portfolio and price data to CSV
    }

    // TODO: Add analysis methods (getTotalValue(), etc.)
}
```

### Step 3: Milestone 2 & Menu (Days 4–8)

#### Main.java (skeleton menu)

```
public class Main {
    public static void main(String[] args) {
        // TODO: Create Portfolio object
        // Menu loop:
        // 1. Run market simulation (ask for days and
marketTrend)
        // 2. Buy or sell shares
        // 3. View portfolio & log
        // 4. Analyze performance (e.g., "Which stock performed
best?")
        // 5. Save & Quit
    }
}
```

Development Log Requirement: Record AI use with prompts and changes. Reflection Prompt: “How did volatility affect your portfolio? What investment strategy did your simulation data suggest?”

### Example 3: Disease Outbreak Simulator

**Theme:** Model how a disease spreads in a population (like real epidemiologists use computers).

**Computational Thinking Focus:** Abstraction (SIR model: Susceptible, Infected, Recovered), simulation rules, parameter experiments, data analysis.

#### Step 1: Design Document (Day 1–2 – submit first!)

**Problem Statement:** Simulate disease spread over days. Users experiment with infection rate, recovery rate, and initial cases. Results saved to CSV for analysis. **Classes:** Person (status: S/I/R), Population (manages ArrayList + simulation). **Pseudocode (you write this):**

```
runSimulation(int days, double infectionRate, double
recoveryRate) {

    for each day {

        calculate newInfections = currentInfected *
infectionRate * (susceptible / total);

        update statuses using loops and if/else;

        log day, susceptible, infected, recovered to file;

    }

}
```

#### Step 2: Milestone 1 – Core Classes (Day 3 – in class)

**Person.java** (skeleton – fill in)

```
public class Person {

    private String status; // "S", "I", or "R"


    public Person(String initialStatus) {

        this.status = initialStatus;

    }

}
```

## APCSA Final Exam Project 2026

```
    public String getStatus() { return status; }

    public void setStatus(String newStatus) { this.status =
newStatus; }

    // TODO: Add any helper methods you designed (e.g.,
canInfect())

}
```

### Population.java (skeleton)

```
import java.util.ArrayList;

import java.io.*;

public class Population {

    private ArrayList<Person> people = new ArrayList<>();

    private String logFile = "outbreak_log.csv";

    public Population(int totalPeople, int initialInfected) {

        // TODO: Create ArrayList with mostly "S" and some "I"

    }

    public void runOneDay(double infectionRate, double
recoveryRate) {

        // TODO: Implement your pseudocode rules here (use loops
+ if/else)

        // Count new infections and recoveries

    }
```



## APCSA Final Exam Project 2026

```
public void saveLog(int day) {  
    // TODO: Append current counts to CSV using PrintWriter  
}  
  
// TODO: Add getter methods for analysis  
(getSusceptibleCount(), etc.)  
}
```

### Step 3: Milestone 2 & Menu (Days 4–8)

Add to Main.java (skeleton):

```
public class Main {  
    public static void main(String[] args) {  
        // TODO: Create Population object  
        // Menu loop:  
        // 1. Run simulation (ask for days, rates)  
        // 2. View log  
        // 3. Analyze (e.g., "On which day did infected peak?")  
        // 5. Quit & save  
    }  
}
```

**Development Log Requirement:** Record every AI prompt you use + your changes.

**Reflection Prompt:** “How did changing the infection rate affect the final recovered population? Why does this matter in real life?”

## Example 4: Book Recommendation System

**Theme:** Build a personal library that recommends books based on user preferences and ratings.

**Computational Thinking Focus:** Data processing, searching/filtering algorithms, recommendation logic (simple scoring), working with real datasets.

### Step 1: Design Document (Day 1–2 – submit first!)

**Problem Statement:** Users add books, rate them, and get personalized recommendations. Program reads a starter library from CSV and saves new ratings. **Classes:** Book (title, author, genre, rating), Library (manages ArrayList + recommendation algorithm). **Pseudocode (you write this):**

```
getRecommendations(String preferredGenre, double minRating) {  
    create new ArrayList for matches;  
    loop through all books {  
        if (genre matches AND rating >= minRating) add to  
matches;  
    }  
    sort matches by rating (or your own scoring algorithm);  
    return top 5;  
}
```

### Step 2: Milestone 1 – Core Classes (Day 3 – in class)

**Book.java** (skeleton)

```
public class Book {  
    private String title;  
    private String author;  
    private String genre;  
    private double rating; // 0.0 to 5.0
```

## APCSA Final Exam Project 2026

```
    public Book(String title, String author, String genre,
double rating) {

        // TODO: Assign values (with validation if you designed
it)

    }

    public String getGenre() { return genre; }

    public double getRating() { return rating; }

    public void setRating(double newRating) { /* TODO */ }


    public String toString() { /* TODO: nice formatted string
*/ }

}
```

### Library.java (skeleton)

```
import java.util.ArrayList;

import java.io.*;

import java.util.Scanner;


public class Library {

    private ArrayList<Book> books = new ArrayList<>();

    private String dataFile = "library.csv";


    public Library() {

        loadFromFile(); // TODO: implement

    }
```

## APCSA Final Exam Project 2026

```
private void loadFromFile() {  
    // TODO: Use Scanner to read CSV and create Book objects  
}  
  
public void addBook(String title, String author, String  
genre) {  
    // TODO: Create new Book and add to ArrayList  
}  
  
public ArrayList<Book> getRecommendations(String genre,  
double minRating) {  
    // TODO: Implement your pseudocode algorithm here  
    return null; // replace with real list  
}  
  
public void saveToFile() {  
    // TODO: Use PrintWriter to save updated library  
}  
}
```

### Step 3: Milestone 2 & Menu (Days 4–8)

**Main.java** (skeleton menu):

```
public class Main {  
    public static void main(String[] args) {  
        Library lib = new Library();  
    }  
}
```

## APCSA Final Exam Project 2026

```
        // Menu loop:

        // 1. Add a new book

        // 2. Rate an existing book (search by title)

        // 3. Get recommendations (ask for genre & min rating)

        // 4. View all books

        // 5. Save & Quit

    }

}
```

**Development Log Requirement:** Every time you use AI, paste the prompt + your modifications.

**Reflection Prompt:** “Which search/filter algorithm did you design? How did testing different minRating values show you the importance of data-driven decisions?”